Assignment 3 extra solution

1. Q1

- **Code**

```
module queue();
    int j = 1;
    int q [$] = {0,2,5};
    int temp;
    initial begin
        q.insert(1, j);
        $display("q after inserting j at index 1 = %p", q);
        q.delete(1);
        $display("q after deleting index 1 = %p", q);
        q.push_front(7);
        $display("q after pushing front value 7= %p", q);
        q.push_back(9);
        $display("q after pushing back value 9= %p", q);
        j = q.pop_back();
        $display("q after poping back = %p", q);
        $display("poped value j = %0d", j);
        j = q.pop_front();
        $display("q after poping front = %p", q);
        $display("poped value j = %0d", j);
        for (int i = 0; i < q.size()/2; i++) begin
            temp = q[i];
            q[i] = q[q.size()-1-i];
            q[q.size()-1-i] = temp;
        end
        $display("q after reversing = %p", q);
        q.sort();
        $display("q after sorting = %p", q);
        q.rsort();
        $display("q after reverse sorting = %p", q);
        q.shuffle();
        $display("q after shuffling = %p", q);
    end

endmodule
```

- **Result**

```
# q after inserting j at index 1 = '{0, 1, 2, 5}
# q after deleting index 1 = '{0, 2, 5}
# q after pushing front value 7= '{7, 0, 2, 5}
# q after pushing back value 9= '{7, 0, 2, 5, 9}
# q after poping back = '{7, 0, 2, 5}
# poped value j = 9
# q after poping front = '{0, 2, 5}
# poped value j = 7
# q after reversing = '{5, 2, 0}
# q after sorting = '{0, 2, 5}
# q after reverse sorting = '{5, 2, 0}
# q after shuffling = '{2, 0, 5}
VSIM 2>
```

2. Q2

- **Package**

```
package adder_pkg;
typedef enum logic signed [3:0] {MAXPOS = 4'd7, MAXNEG = -4'd8, ZERO = 0} val_t;

class rand_adder;

    rand bit reset;
    rand logic signed [3:0] A,B;
    bit clk;

    constraint constraints {
        reset dist {0:=99, 1:=1};
        A dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, [-7:-1]:=20, [1:6]:=20};
        B dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, [-7:-1]:=20, [1:6]:=20};
    }

    covergroup Covgrp_A;
        A_cp: coverpoint A {
            bins data_0 = {0};
            bins data_max = {MAXPOS};
            bins data_min = {MAXNEG};
            bins data_default = default;
        }

        A_transition_cp: coverpoint A {
            bins data_0max = {ZERO, MAXPOS};
            bins data_maxmin = {MAXPOS, MAXNEG};
            bins data_minmax = {MAXNEG, MAXPOS};
        }
    endgroup

    covergroup Covgrp_B;
        B_cp: coverpoint B {
            bins data_0 = {0};
            bins data_max = {MAXPOS};
            bins data_min = {MAXNEG};
            bins data_default = default;
        }

        B_transition_cp: coverpoint B {
            bins data_0max = {ZERO, MAXPOS};
            bins data_maxmin = {MAXPOS, MAXNEG};
            bins data_minmax = {MAXNEG, MAXPOS};
        }
    endgroup
      function new();
          Covgrp_A = new();
          Covgrp_B = new();
      endfunction

endclass
endpackage : adder_pkg
```

- **testbench**

```
import adder_pkg::*;    Could not find t
module adder_tb ();
  //? Interface signals
  logic clk, reset;
  logic signed [3:0] A, B;
  logic signed [4:0] C;
  //? Variables
  int error_count, correct_count;
  rand_adder addr_obj;
  //? DUT Instantiation
  adder DUT (
      .clk(clk),
      .reset(reset),
      .A(A),
      .B(B),
      .C(C)
  );
  //? CLK generator
    initial begin
    clk = 0;
    forever begin
      #1 clk = ~clk;
      addr_obj.clk = clk;
    end
  end
  //? Sampling logic
  always @(posedge reset) begin
    addr_obj.Covgrp_A.stop();
    addr_obj.Covgrp_B.stop();
  end

  always @(negedge reset) begin
    addr_obj.Covgrp_A.start();
    addr_obj.Covgrp_B.start();
  end

  always @(posedge clk) begin
    addr_obj.Covgrp_A.sample();
    addr_obj.Covgrp_B.sample();
  end
```

```
  initial begin
    //? Initialization part
    addr_obj = new();
    clk = 0;
    reset = 0;
    A = 0;
    B = 0;
    error_count = 0;
    correct_count = 0;
    @(negedge clk);
    assert_reset();
    //? Randomization test
    for(int i=0; i<100; i++) begin
      assert (addr_obj.randomize());
      A = addr_obj.A;
      B = addr_obj.B;
      reset = addr_obj.reset;
      if (!reset) check_result(A+B);
      else check_result(0);

    end
    $display("error_counter=%d, correct_counter=%d", error_count, correct_count);
    $stop;
  end
  //? TASKS
  task assert_reset();
    reset = 1;
    check_result(0);
    reset = 0;
  endtask

  task check_result(input [4:0] expected_sum);
    @(negedge clk);
    if (expected_sum == C) correct_count++;
    else begin
      error_count++;
      $display("xxxx ERROR: the actual output result is not matching the expected result :
      $display("A=%b ,B=%b, C_exp=%b, C_actual=%b", A, B, expected_sum, C);
    end
  endtask
endmodule
```

- **Code coverage**

❖ **Branch coverage**

```
Coverage Report by DU with details

================================================================================
=== Design Unit: work.adder
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         2         2         0   100.00%

================================Branch Details================================
```

❖ **Statement coverage**

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       3         3         0   100.00%

================================Statement Details================================

Statement Coverage for Design Unit work.adder --
```

❖ **Toggle coverage**

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

================================Toggle Details================================

Toggle Coverage for Design Unit work.adder

                                    Node      1H->0L      0L->1H  "Coverage"
                                    ----------------------------------------
                                  A[0-3]           1           1      100.00
                                  B[0-3]           1           1      100.00
                                  C[0-4]           1           1      100.00
                                     clk           1           1      100.00
                                   reset           1           1      100.00

Total Node Count     =         15
Toggled Node Count   =         15
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (30 of 30 bins)


Total Coverage By Design Unit (filtered view): 100.00%
```

- **Functional coverage**

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_ins
/adder_pkg/rand_...		100.00%						
TYPE Covgrp_...		100.00%	100	100.00...		✓	auto(1)	
TYPE Covgrp_...		100.00%	100	100.00...		✓	auto(1)	
CVP Covgr...		100.00%	100	100.00...		✓		
bin data...		15	1	100.00...		✓		
bin data...		15	1	100.00...		✓		
bin data...		19	1	100.00...		✓		
default...		51	-	-		✓		
CVP Covgr...		100.00%	100	100.00...		✓		
bin data...		30	1	100.00...		✓		
bin data...		34	1	100.00...		✓		
bin data...		34	1	100.00...		✓		

- **Simulation result**

```
# Loading work.adder(fast)
# error_counter=          0, correct_counter=        101
# ** Note: $stop    : adder_tb.sv(63)
#    Time: 204 ns  Iteration: 1  Instance: /adder_tb
# Break in Module adder_tb at adder_tb.sv line 63
```

3. Q3

- **Package**

```
package FSM_PACKAGE;
    typedef enum logic [1:0] { IDLE, ZERO, ONE, STORE } state_e;
    class fsm_transaction;
        //? variables
        rand bit rst;
        rand bit x;
        state_e state;
        bit y_exp;
        bit [9:0] user_count_exp;
        bit clk;
        //? constraints
        constraint c_dist {
            rst  dist {0:=98, 1:=2};
            x    dist {0:=67, 1:=33};
        }
        //? functional coverage
        covergroup Covgrp_X @(posedge clk);
            X_cp: coverpoint x {
                bins x_tr = (0 => 1 => 0);
            }
        endgroup
        //? constructor
        function new();
            rst = 0;
            x = 0;
            y_exp = 0;
            user_count_exp = 0;
            state = IDLE;
            Covgrp_X = new();
        endfunction
    endclass
endpackage
```

- **Testbench**

```
import FSM_PACKAGE::*;    Could not find the package (FSM_PACKAGE). Design read will c
module FSM_tb ();
  //? local parameters
  localparam CLK_CYCLE = 2;
  //? class object handler creation
  fsm_transaction fsm_obj;
  //? Interface signals
  logic rst, clk, x, y;
  logic [9:0] users_count;
  //? Variables
  int error_count, correct_count;
  //? DUT instantiation
  FSM_010 DUT_FSM (
      .clk(clk),
      .rst(rst),
      .x(x),
      .y(y),
      .users_count(users_count)
  );
  //? CLK generation
  initial begin
    clk = 0;
    forever begin
      #1 clk = ~clk;
      fsm_obj.clk = clk;
    end
  end
  //? Driver block
  initial begin
    //* initialization
    clk = 0;
    fsm_obj = new();
    //* reset test
    fsm_obj.rst = 1;
    rst = fsm_obj.rst;
    check_result();
    //* randomization test
    for (int i = 0; i < 100; i++) begin
      assert(fsm_obj.randomize());
      x   = fsm_obj.x;
      rst = fsm_obj.rst;
      check_result();
    end
    $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
    $stop;
  end
  //? TASKS
```

```
  //? TASKS
  task check_result();
    logic y_exp;
    logic [9:0] users_count_exp;
    golden_model(y_exp, users_count_exp);
    #CLK_CYCLE;
    if (y_exp == y && users_count_exp == users_count) correct_count++;
    else begin
      error_count++;
      $display(
          "xxxx ERROR at time %0t: the actual output result is not matching the expected result xxxx",
          $time);
      $display("y=%b ,users_count=%0d, y_exp=%b, users_count_exp=%0d", y, users_count, y_exp,
               users_count_exp);
    end
  endtask

  task golden_model(output y_exp, output [9:0] count_exp);
    if (rst) begin
      fsm_obj.y_exp = 0;
      fsm_obj.user_count_exp = 0;
      fsm_obj.state = IDLE;    Undefined variable: 'IDLE'.
    end else begin
      case (fsm_obj.state)
        IDLE: if (x) fsm_obj.state = IDLE; else fsm_obj.state = ZERO;    Undefined variable: 'IDLE'.
        ZERO: if (x) fsm_obj.state = ONE; else fsm_obj.state = ZERO;    Undefined variable: 'ONE'.
        ONE: if (x) fsm_obj.state = IDLE; else fsm_obj.state = STORE;    Undefined variable: 'STORE'.
        STORE: begin
          fsm_obj.user_count_exp++;
          if (x) fsm_obj.state = IDLE; else fsm_obj.state = ZERO;
        end
        default: fsm_obj.state = IDLE;
      endcase
      if(fsm_obj.state == STORE) fsm_obj.y_exp = 1; else fsm_obj.y_exp = 0;
    end
    y_exp = fsm_obj.y_exp;
    count_exp = fsm_obj.user_count_exp;
  endtask
endmodule
```

- ## Code coverage

❖ Branch coverage

```
Coverage Report by instance with details

================================================================================
=== Instance: /\FSM_tb#DUT_FSM
=== Design Unit: work.FSM_010
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        20        20         0   100.00%

================================Branch Details================================
```

❖ Statement coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      16        16         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\FSM_tb#DUT_FSM  --
```

❖ Toggle coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         28        28         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\FSM_tb#DUT_FSM  --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                           clk           1           1      100.00
                                        cs[1-0]          1           1      100.00
                                        ns[1-0]          1           1      100.00
                                           rst           1           1      100.00
                                  users_count[5-0]       1           1      100.00
                                             x           1           1      100.00
                                             y           1           1      100.00

Total Node Count     =         14
Toggled Node Count   =         14
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (28 of 28 bins)


Total Coverage By Instance (filtered view): 100.00%
```

- ## Functional coverage

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances
/FSM_PACKAGE/fsm_transaction		100.00%					
TYPE Covgrp_X		100.00%	100	100.00...		✓	auto(0)
CVP Covgrp_X::X_cp		100.00%	100	100.00...		✓	
INST \/FSM_PACKAGE::fsm...		100.00%	100	100.00...		✓	
CVP X_cp		100.00%	100	100.00...		✓	
bin x_tr		13	1	100.00...		✓	

- ## Simulation results

```
# Loading work.FSM_010(fast)
# error_counts = 0, correct_counts = 101
# ** Note: $stop    : FSM_tb.sv(45)
#    Time: 202 ns  Iteration: 0  Instance: /FSM_tb
# Break in Module FSM_tb at FSM_tb.sv line 45
```

4. Q4

- **Code**

```
module mem();

    typedef logic [19:0] addr_t;
    logic [23:0] memory [addr_t];

    initial begin
        memory [0] = 24'hA50400;
        memory [20'h400] = 24'h123456;
        memory [20'h401] = 24'h789ABC;
        memory [20'hFFFFF] = 24'h0F1E2D;
        // Print number of elements
        $display("Number of elements = %0d", memory.num());

        // Iterate and print all elements
        foreach (memory[i]) begin
            $display("memory[%0h] = %0h", i, memory[i]);
        end
        $stop;
    end

endmodule
```

- **results**

```
# Loading work.mem(fast)
# Number of elements = 4
# memory[0] = a50400
# memory[400] = 123456
# memory[401] = 789abc
# memory[fffff] = f1e2d
# ** Note: $stop    : mem.sv(18)
#    Time: 0 ns  Iteration: 0  Instance: /mem
# Break in Module mem at mem.sv line 18
```

5. Q5

- **Code**

```
module packet_struct;

    typedef logic [6:0] seven_t;

    typedef struct {
        seven_t CRC;
        seven_t DATA;
        seven_t CMD;
        seven_t HEADER;
    } packet;

    packet my_packet;

    initial begin
        my_packet.HEADER = 7'h5A;
        $display("HEADER = %0h", my_packet.HEADER);
    end

endmodule
```

- **Results**

```
# Loading sv_std.std
# Loading work.packet_struct(fast)
# HEADER = 5a
VSIM 2>
```